

Protocolo de Comunicacion SIQP/1.0

Propuesta Proyecto Entregable 1

CI-0123 Proyecto Integrador de Redes y Sistemas Operativos

Grupo 2 — Equipo 2 — 2026-II

John Vargas Arroyo – C4K672

Fabricio Padilla Madrigal – C35837

Andres Zamora Orozco – C4L163

Wilson Chavarria Miranda – C22114

Profs. Francisco Arroyo, Amanda Alpizar

26 de agosto de 2026

Índice

1. Protocolo SIQP/1.0	3
2. Componentes del sistema	3
3. Comandos expuestos al usuario	3
3.1. Formato de la respuesta de info	4
4. Implementacion en la fase 1	5
5. Implementacion en fases posteriores	5
6. Formato general de los mensajes SIQP/1.0	5
6.1. Estructura del campo CUERPO	6
6.2. Escape de caracteres reservados	6
6.3. Reglas generales del protocolo	6
6.4. Organizacion de los tipos de mensaje	7
6.5. Tipos de mensaje del protocolo	7
6.6. Tipos de error del protocolo	9
6.7. Ejemplos	9
7. Identificacion de nodos	10
8. Comunicacion entre intermediarios	11
8.1. Encapsulamiento de consultas remotas	11
8.2. Tipos de mensaje necesarios	11
8.3. Relacion con los mensajes basicos	12
8.4. Eventos que deben propagarse	12
8.5. Reglas de funcionamiento	13
9. Manejo de la tabla de rutas	13
9.1. Estructura propuesta	14
9.2. Reglas	14

9.3. Ejemplo de tabla de rutas	14
10. Manejo de la bitacora de eventos	15
10.1. Estructura de cada entrada	15
10.2. Mensajes para consulta de bitacora	15
11. Transporte por tipo de mensaje	16
12. Tabla de instrucciones final	16
13. Casos de prueba	17

1. Protocolo SIQP/1.0

Para la comunicacion interna entre los componentes del sistema se propone el protocolo SIQP/1.0 (Store Inventory Query Protocol). Este corresponde a un protocolo textual disenado para facilitar la coordinacion entre el servidor intermediario y los servidores de inventario que administran las categorias y productos de la tienda. Su objetivo principal es permitir el registro de servidores de inventario ante un intermediario, asi como su eliminacion o desconexion cuando sea necesario. Ademias, contempla la consulta del conjunto de comandos disponibles, la consulta del catalogo de categorias, la consulta de los productos contenidos en una categoria, la consulta de la cantidad en existencia de un producto y la consulta de su ficha completa, entendida como la cantidad, el precio, la ubicacion fisica dentro de la tienda y una descripcion breve del articulo. Igualmente contempla el manejo de errores y la generacion de eventos de bitacora para monitoreo y depuracion.

El protocolo SIQP/1.0 se utiliza como mecanismo de comunicacion interno del sistema, independiente de la interfaz expuesta al usuario. Asimismo, permite modelar la interaccion entre los componentes tanto en la implementacion distribuida como en la simulacion local mediante hilos.

2. Componentes del sistema

El sistema se organiza en tres componentes principales: cliente, intermediario y servidor de inventario. El cliente representa al usuario final y es el encargado de traducir las preguntas escritas por la persona a solicitudes concretas sobre el catalogo. El intermediario corresponde al componente coordinador del sistema, su funcion principal es recibir solicitudes del cliente, determinar a que servidor de inventario debe consultar y devolver una respuesta final procesada. El servidor de inventario es el componente responsable de almacenar la informacion de las categorias y productos en su propio sistema de almacenamiento y responder a las consultas que recibe.

En el sistema se emplean dos mecanismos de comunicacion diferenciados. La comunicacion entre el cliente y el intermediario se realiza mediante el protocolo HTTP, el cual actua como interfaz externa del sistema. Por otro lado, la comunicacion entre el intermediario y los servidores de inventario se realiza mediante el protocolo SIQP/1.0, que define el formato y la semantica de los mensajes intercambiados internamente.

3. Comandos expuestos al usuario

La interfaz que percibe la persona que usa el sistema esta compuesta por cinco comandos. Estos comandos no forman parte del protocolo interno, sino que constituyen la capa externa que el cliente traduce a mensajes SIQP/1.0. La correspondencia entre ambas capas se define en el siguiente cuadro, y es la que determina el conjunto minimo de tipos de mensaje que el protocolo debe soportar.

Comando de usuario	Resultado esperado	Mensajes SIQP/1.0
help	Retorna la lista de comandos que reconoce el sistema, cada uno acompañado de la descripcion en espanol de lo que hace.	10 HELP_QUERY → 11 HELP_RESULT
inventory	Retorna todas las categorias que maneja la tienda.	12 CATEGORY_QUERY → 13 CATEGORY_RESULT

Comando de usuario	Resultado esperado	Mensajes SIQP/1.0
<code>have <categoria></code>	Retorna los productos que hay dentro de esa categoria.	14 PRODUCT_QUERY → 15 PRODUCT_RESULT
<code>stock <cat> <prod></code>	Retorna cuantas unidades hay de ese producto.	16 STOCK_QUERY → 17 STOCK_RESULT
<code>info <cat> <prod></code>	Retorna la ficha del producto, es decir su cantidad, precio, ubicacion y descripcion breve, separadas por punto y coma.	18 INFO_QUERY → 19 INFO_RESULT

Los nombres de los comandos se escriben en ingles, mientras que todo el texto que el sistema devuelve al usuario se presenta en espanol. Esto aplica en particular al comando **help**, cuya respuesta contiene el nombre de cada comando junto con su explicacion en espanol, de modo que la persona que usa el sistema pueda descubrir la sintaxis sin recurrir a documentacion externa.

3.1. Formato de la respuesta de info

El comando **info** agrupa dos consultas que en la version anterior de esta propuesta eran independientes, las de precio y ubicacion, e incorpora ademas la cantidad y una descripcion breve del producto. La razon de agruparlas es que los cuatro valores se obtienen de una misma lectura del bloque de almacenamiento, de manera que resolverlas por separado multiplicaria los accesos sin aportar informacion adicional. El comando **stock** se conserva aparte porque responde la pregunta mas frecuente del inventario con un solo valor y sin obligar al cliente a descartar el resto de la ficha.

La respuesta se entrega al usuario como una unica linea cuyos campos se separan con punto y coma, en un orden fijo, de manera que el cliente pueda dividirla sin necesidad de interpretar etiquetas:

`cantidad;precio;ubicacion;descripcion`

Por ejemplo, la consulta `info electronica laptop` produce la linea siguiente:

`5;$15000.00;Bodega, Seccion Electronica;computadora portatil para trabajo y estudio`

El orden de los campos forma parte del contrato del comando y no debe alterarse, ya que el cliente los identifica por posicion y no por nombre. Un campo sin valor se entrega vacio pero conserva el punto y coma que lo delimita, de modo que la cantidad de separadores sea siempre la misma y la division de la linea nunca dependa del contenido. Dado que el punto y coma actua como separador, la descripcion almacenada no puede contener ese caracter; la restriccion se aplica al momento de cargar el catalogo en el sistema de archivos. El simbolo de moneda no se almacena ni viaja por la red: lo agrega el cliente al componer la linea, a partir del valor numerico que entrega el protocolo.

Dentro del protocolo la ficha no viaja como una unica cadena, sino como cuatro atributos independientes del campo **CUERPO**, el cual ya utiliza el punto y coma como separador de atributos. El cliente descarta las etiquetas y conserva unicamente los valores para construir la linea que ve el usuario. Esta separacion permite que el intermediario procese la respuesta sin depender de posiciones fijas y que el formato final se arme en un unico punto del sistema.

Los cinco comandos se traducen a rutas HTTP en la interfaz externa de la siguiente forma:

```
GET /help
GET /inventory
GET /have/<categoria>
GET /stock/<categoria>/<producto>
GET /info/<categoria>/<producto>
```

4. Implementacion en la fase 1

En la primera fase del proyecto, la interaccion del sistema se implementa de forma simplificada, estableciendo comunicacion directa entre el cliente y el servidor de inventario mediante HTTP. En este escenario, el cliente es responsable de interpretar la pregunta escrita por el usuario, extraer de ella la categoria y el producto cuando corresponda, y realizar la solicitud necesaria para obtener la informacion. El servidor de inventario responde con el dato puntual solicitado, y el cliente lo presenta al usuario en texto plano, sin incluir elementos HTML. Conviene notar que la ficha completa de un producto se obtiene con una sola solicitud gracias al comando `info`, por lo que el cliente no necesita combinar respuestas parciales para construir el resultado que muestra.

Adicionalmente, en esta fase se define y simula la comunicacion entre el cliente, el servidor de inventario y un intermediario mediante el protocolo SIQP/1.0, utilizando hilos comunicados mediante memoria compartida. Esta simulacion permite validar el formato de los mensajes, el manejo de solicitudes y respuestas, ademas de la logica general del sistema sin requerir aun una implementacion distribuida completa.

5. Implementacion en fases posteriores

Para las etapas posteriores del proyecto, se propone una arquitectura en la que el cliente se comunica exclusivamente con el intermediario mediante HTTP. En este contexto, el intermediario actua como un adaptador entre la interfaz externa y el protocolo interno, interpretando las solicitudes del cliente, traduciendo a mensajes SIQP/1.0 y enviandolas a los servidores de inventario correspondientes. Una vez recibidas las respuestas, el intermediario procesa la informacion y genera una respuesta final en formato HTTP que contiene unicamente los datos relevantes para el cliente. De esta forma, el sistema queda organizado en capas de comunicacion diferenciadas, donde HTTP se utiliza para la interaccion con el usuario y SIQP/1.0 para la coordinacion interna. Esta separacion permite la incorporacion de multiples servidores de inventario y mecanismos adicionales como tablas de rutas en fases mas avanzadas.

6. Formato general de los mensajes SIQP/1.0

El protocolo SIQP/1.0 define un formato textual estructurado para todos los mensajes intercambiados entre los componentes internos del sistema. Cada mensaje esta compuesto por una secuencia de campos separados por el caracter `|`, siguiendo la estructura general mostrada a continuacion:

```
SIQP/1.0|ID_TIPO|ID_MENSAJE|ORIGEN|DESTINO|CUERPO
```

Cada uno de los campos tiene un significado especifico dentro del protocolo:

Campo	Descripcion
SIQP/1.0	Version del protocolo. Permite validar compatibilidad entre nodos.
ID_TIPO	Tipo de mensaje que se esta enviando (por ejemplo: REGISTER, INFO_QUERY, ACK, ERROR).
ID_MENSAJE	Identificador unico de la solicitud, utilizado para relacionar solicitudes con sus respectivas respuestas.
ORIGEN	Identificador del nodo que envia el mensaje (cliente, intermediario o servidor de inventario).
DESTINO	Identificador del nodo al que va dirigido el mensaje.
CUERPO	Contiene los datos especificos del mensaje, estructurados en formato clave-valor. Puede variar segun el tipo de mensaje.

6.1. Estructura del campo CUERPO

El campo CUERPO utiliza una estructura basada en atributos, definida mediante los siguientes separadores:

Simbolo	Uso
;	Separador entre atributos
=	Asignacion clave=valor
,	Separador de elementos dentro de listas
\	Caracter de escape para valores que contienen simbolos reservados

6.2. Escape de caracteres reservados

A diferencia de un catalogo de figuras, los valores manejados por este sistema incluyen texto libre. En particular, los atributos **ubicacion** y **descripcion** contienen frases como **Bodega**, **Seccion Electronica**, que incorporan comas y por lo tanto colisionan con el separador de listas. Por esta razon se establece que los caracteres reservados |, ;, =, , y \ deben escribirse precedidos de una barra invertida cuando aparecen dentro de un valor. El receptor debe deshacer el escape antes de interpretar el dato. Un valor que contenga un caracter reservado sin escapar constituye un mensaje mal formado y debe responderse con el error correspondiente.

El atributo **descripcion** constituye la unica excepcion a esta regla, ya que no puede contener el caracter ; en ninguna forma, ni siquiera escapado. La razon es que ese simbolo separa los campos de la linea que el comando **info** entrega al usuario, de modo que permitirlo dentro del texto haria ambigua esa division aunque el mensaje del protocolo fuese valido.

6.3. Reglas generales del protocolo

Para garantizar la consistencia del protocolo SIQP/1.0, se establecen las siguientes reglas generales.

Todo mensaje debe respetar exactamente la estructura de seis campos definida por el protocolo. Si un mensaje llega con campos faltantes, separadores invalidos o estructura incorrecta, el receptor debe generar una respuesta de error indicando que el formato no es valido.

Toda solicitud debe generar una unica respuesta asociada. Esta respuesta puede corresponder a un mensaje especifico del tipo de operacion, a un mensaje de confirmacion (**ACK**) o a un mensaje de error (**ERROR**). En caso de no recibirse respuesta dentro del tiempo definido por la implementacion, la operacion se considera fallida por **TIMEOUT**.

Toda respuesta debe reutilizar el mismo **ID_MENSAJE** de la solicitud original, de manera que ambos mensajes queden correlacionados como parte de la misma operacion logica.

El campo **CUERPO** puede quedar vacio en aquellos mensajes cuyo tipo no requiere datos adicionales. Sin embargo, el resto de los campos debe mantenerse siempre presente.

El protocolo es sensible a la version indicada en el primer campo. Si un nodo recibe un mensaje con una version no compatible, debe responder mediante un mensaje de error.

El valor de **ID_TIPO** debe corresponder a uno de los codigos oficialmente definidos por el protocolo. Si el receptor recibe un identificador de tipo desconocido, debe responder con un error apropiado.

El nombre del comando y los nombres de categoria y producto se tratan sin distincion entre mayusculas y minusculas y se normalizan a minusculas antes de compararse, de manera que lo que escriba el usuario no dependa de como haya sido tecleado.

La ausencia de un recurso no se considera un error de protocolo. Que una categoria o un producto no existan es una respuesta legitima a una consulta bien formada, por lo que se representa mediante los tipos de respuesta 30 y 31 y no mediante el rango de errores. El rango de errores queda reservado para fallas de comunicacion, de formato o de estado del sistema.

6.4. Organizacion de los tipos de mensaje

Los tipos de mensaje de SIQP/1.0 se representan mediante un identificador numerico (**ID_TIPO**). Cada identificador corresponde a una operacion o respuesta especifica dentro del sistema. De esta forma, el protocolo mantiene un formato uniforme y permite que cada componente interprete los mensajes a partir de una tabla de equivalencias previamente definida.

Los identificadores se agrupan por rangos segun su funcion, lo cual facilita la interpretacion, el procesamiento y la extension del protocolo. El rango 01 a 09 corresponde al control y gestion de nodos, que permite registrar servidores de inventario, retirarlos del sistema, verificar que continuan activos y consultar la bitacora. El rango 10 a 19 corresponde a las consultas de negocio y sus respuestas, es decir, a los cinco comandos expuestos al usuario, mientras que el rango 20 a 29 queda reservado para las consultas que se agreguen en versiones posteriores. El rango 30 a 39 agrupa las respuestas que indican ausencia de un recurso. El rango 40 a 59 se reserva para la comunicacion entre intermediarios. El rango 90 a 99 corresponde a las confirmaciones y el rango 100 en adelante a los errores. Esta estructura permite identificar rapidamente el tipo de operacion que representa un mensaje, simplifica la validacion en la implementacion y facilita la incorporacion de nuevas funcionalidades sin afectar la coherencia del protocolo.

6.5. Tipos de mensaje del protocolo

ID	Significado	Naturaleza	Proposito	Respuesta esperada
01	REGISTER	Solicitud	Registrar un servidor de inventario en el sistema.	90/Error

ID	Significado	Natura- leza	Proposito	Respuesta esperada
02	UNREGISTER	Solicitud	Retirar un servidor de inventario del sistema.	90/Error
03	HEARTBEAT	Solicitud	Verificar que un nodo sigue activo.	90/Error
04	ROUTE_UPDATE	Solicitud	Notificar cambios en rutas, nodos o disponibilidad remota.	90/Error
05	ROUTE_REQUEST	Solicitud	Solicitar el estado actual de rutas a otro intermediario.	06/Error
06	ROUTE_SNAPSHOT	Respuesta	Devolver una copia del estado actual de rutas conocidas.	Nada
07	EVENT_LOG_QUERY	Solicitud	Consultar la bitacora de un nodo.	08/Error
08	EVENT_LOG_RESULT	Respuesta	Devolver eventos registrados.	Nada
10	HELP_QUERY	Solicitud	Consultar el conjunto de comandos que reconoce el nodo.	11/Error
11	HELP_RESULT	Respuesta	Devolver la lista de comandos disponibles.	Nada
12	CATEGORY_QUERY	Solicitud	Consultar el catalogo de categorias disponibles.	13/Error
13	CATEGORY_RESULT	Respuesta	Devolver la lista de categorias.	Nada
14	PRODUCT_QUERY	Solicitud	Consultar los productos de una categoria.	15,30/Error
15	PRODUCT_RESULT	Respuesta	Devolver la lista de productos de la categoria.	Nada
16	STOCK_QUERY	Solicitud	Consultar la cantidad en existencia de un producto.	17,30,31/Error
17	STOCK_RESULT	Respuesta	Devolver la cantidad en existencia.	Nada
18	INFO_QUERY	Solicitud	Consultar la ficha completa de un producto.	19,30,31/Error
19	INFO_RESULT	Respuesta	Devolver cantidad, precio, ubicacion y descripcion.	Nada
30	CATEGORY_NOT_FOUND	Respuesta	Indica que la categoria consultada no existe.	Nada
31	PRODUCT_NOT_FOUND	Respuesta	Indica que el producto no existe en esa categoria.	Nada
90	ACK	Respuesta	Confirmacion de exito sin contenido adicional.	Nada

6.6. Tipos de error del protocolo

ID	Significado	Descripcion	Cuando pasa
101	INVALID_FORMAT	El mensaje no cumple con la estructura del protocolo.	Campos faltantes, separadores incorrectos, escape invalido.
102	DESTINATION_UNAVAILABLE	El nodo destino no existe o no es alcanzable.	Nodo desconocido o desconectado.
103	UNSUPPORTED_VERSION	La version del protocolo no es valida.	No es SIQP/1.0.
104	TIMEOUT	No se recibio respuesta en el tiempo esperado.	Falla de comunicacion o nodo caido.
105	UNSUPPORTED_TYPE	El ID_TIPO no es reconocido.	Codigo inexistente.
106	INVALID_PARAMETERS	El cuerpo del mensaje es incorrecto.	Falta categoria, falta producto, clave desconocida.
107	NODE_NOT_REGISTERED	El nodo no esta registrado en el sistema.	No se ha hecho REGISTER .
108	OPERATION_NOT_ALLOWED	Operacion invalida en el estado actual.	Por ejemplo, consultar sin registro previo.
109	STORAGE_ERROR	El nodo no pudo leer su medio de almacenamiento.	Bloque corrupto, puntero invalido o cadena rota en el archivo de datos.

El error 109 se agrega respecto de propuestas anteriores porque en este sistema el servidor de inventario resuelve cada consulta leyendo bloques de un archivo de datos propio. Una falla de lectura no es una ausencia de recurso ni un problema de formato, y debe poder distinguirse de ambas situaciones tanto para el cliente como para la bitacora.

6.7. Ejemplos

Registro exitoso de un servidor de inventario:

```
SIQP/1.0|01|00001|SI-01|INT-01|categorias=ropa,comida,electronica,juguetes
SIQP/1.0|90|00001|INT-01|SI-01|
```

Consulta de comandos disponibles, correspondiente a **help**. Las comas contenidas en las descripciones viajan escapadas:

```
SIQP/1.0|10|00010|CLI-01|INT-01|
SIQP/1.0|11|00010|INT-01|CLI-01|help=Muestra los comandos disponibles y
que hace cada uno;inventory=Muestra las categorias de la tienda;
have=Muestra los productos de una categoria;stock=Muestra cuantas unidades
hay de un producto;info=Muestra cantidad\, precio\, ubicacion y descripcion
de un producto
```

Consulta de categorias, correspondiente a `inventory`:

```
SIQP/1.0|12|00011|INT-01|SI-01|  
SIQP/1.0|13|00011|SI-01|INT-01|categorias=ropa,comida,electronica,juguetes
```

Consulta de productos de una categoria, correspondiente a `have ropa`:

```
SIQP/1.0|14|00012|INT-01|SI-01|categoria=ropa  
SIQP/1.0|15|00012|SI-01|INT-01|categoria=ropa;productos=camisa,pantalon,zapatos
```

Consulta de cantidad en existencia, correspondiente a `stock comida arroz`:

```
SIQP/1.0|16|00013|INT-01|SI-01|categoria=comida;producto=arroz  
SIQP/1.0|17|00013|SI-01|INT-01|categoria=comida;producto=arroz;cantidad=100
```

Consulta de la ficha completa, correspondiente a `info electronica laptop`. Se observa el escape de la coma contenida en la ubicacion:

```
SIQP/1.0|18|00014|INT-01|SI-01|categoria=electronica;producto=laptop  
SIQP/1.0|19|00014|SI-01|INT-01|categoria=electronica;producto=laptop;  
cantidad=5;precio=15000.00;ubicacion=Bodega\, Seccion Electronica;  
descripcion=computadora portatil para trabajo y estudio
```

A partir de esa respuesta, el cliente descarta las etiquetas, deshace el escape y presenta al usuario la siguiente linea:

```
5;$15000.00;Bodega, Seccion Electronica;computadora portatil para  
trabajo y estudio
```

Categoria inexistente:

```
SIQP/1.0|14|00016|INT-01|SI-01|categoria=libros  
SIQP/1.0|30|00016|SI-01|INT-01|categoria=libros
```

Producto inexistente dentro de una categoria valida:

```
SIQP/1.0|18|00017|INT-01|SI-01|categoria=comida;producto=pan  
SIQP/1.0|31|00017|SI-01|INT-01|categoria=comida;producto=pan
```

Error de formato invalido:

```
SIQP/1.0|101|00018|SI-01|INT-01|
```

7. Identificacion de nodos

Cada nodo dentro del sistema posee un identificador unico que se utiliza en los campos `ORIGEN` y `DESTINO` de los mensajes del protocolo SIQP/1.0. Se propone un prefijo que indica el tipo de nodo seguido de un identificador numerico o logico. Por ejemplo, `CLI-01` corresponde a un cliente, `INT-01` a un servidor intermedio y `SI-01` a un servidor de inventario.

Este esquema permite distinguir facilmente el rol de cada componente dentro del sistema, facilitando el enrutamiento de mensajes y el registro de eventos. Durante el proceso de registro, el servidor de inventario debe incluir en el campo `CUERPO` la lista de categorias que administra, ya que esa lista es la unidad de enrutamiento del sistema: el intermediario decide a que nodo dirigir una consulta a partir de la categoria mencionada por el usuario, sin necesidad de conocer de antemano los productos individuales.

8. Comunicacion entre intermediarios

La comunicacion entre servidores intermedios tiene dos funciones principales dentro del sistema. La primera es la sincronizacion de estado, es decir, propagar informacion sobre que servidores de inventario estan activos, que categorias manejan y que cambios ocurren en su disponibilidad. La segunda es la resolucion de consultas remotas, ya que un intermediario solo puede acceder a los servidores de inventario de su propia isla, por lo tanto, cuando necesita un recurso que pertenece a otra isla, debe solicitarlo al intermediario responsable de esa isla, y este sera quien consulte localmente a su servidor de inventario y devuelva el resultado. Bajo este modelo, un intermediario no consulta directamente a un servidor de inventario remoto. La tabla de rutas no solo indica que servidor posee una categoria, sino tambien que intermediario debe ser contactado para obtenerla.

Los mensajes **REGISTER** y **UNREGISTER** solo se usan para la comunicacion del servidor de inventario con el intermediario local, no para comunicacion entre intermediarios. Entre intermediarios no se registra un servidor ajeno, en cambio lo que se hace es anunciarlo o propagar su estado mediante **ROUTE_UPDATE**. Cuando un servidor de inventario se registra en una isla, su intermediario local lo acepta con **REGISTER**, lo agrega a su tabla local y luego notifica el cambio a los demas mediante **ROUTE_UPDATE**. Los otros intermediarios actualizan sus tablas remotas y responden con **ACK**.

Cuando un servidor intermedio recibe una consulta sobre una categoria o un producto, debe intentar resolverla primero utilizando los servidores de inventario registrados en su propia isla. Si la categoria solicitada no se encuentra disponible localmente, entonces debe consultar su tabla de rutas para identificar que intermediario remoto administra el recurso requerido y enviarle la solicitud correspondiente. De esta manera, el sistema prioriza la resolucion local y utiliza la comunicacion entre intermediarios unicamente cuando el recurso no puede obtenerse dentro de la propia isla.

8.1. Encapsulamiento de consultas remotas

Para evitar duplicar cada uno de los cinco tipos de consulta en una variante remota, se propone que las consultas entre intermediarios utilicen un mecanismo de encapsulamiento. El intermediario que no puede resolver una consulta localmente construye un mensaje 40 **REMOTE_QUERY** cuyo cuerpo contiene el mensaje **SIQP** original completo, escapado segun las reglas ya definidas. El intermediario remoto lo desencapsula, lo resuelve localmente como cualquier otra consulta y devuelve la respuesta encapsulada en un mensaje 41 **REMOTE_RESULT**.

Esta decision mantiene el numero de tipos de mensaje acotado y garantiza que cualquier consulta que se agregue al protocolo en el futuro sea automaticamente enrutable entre islas sin definir tipos nuevos. La alternativa, consistente en declarar un par de mensajes remotos por cada consulta local, obligaria a modificar la tabla de instrucciones cada vez que se agregue un comando.

8.2. Tipos de mensaje necesarios

ID	Significado	Naturaleza	Proposito	Respuesta esperada
04	ROUTE_UPDATE	Solicitud	Notificar cambios en rutas, nodos o disponibilidad remota.	90/Error
05	ROUTE_REQUEST	Solicitud	Solicitar el estado actual de rutas a otro intermediario.	06/Error

ID	Significado	Naturaleza	Proposito	Respuesta esperada
06	ROUTE_SNAPSHOT	Respuesta	Devolver una copia del estado actual de rutas conocidas.	Nada
40	REMOTE_QUERY	Solicitud	Encapsular una consulta local dirigida a otra isla.	41/Error
41	REMOTE_RESULT	Respuesta	Devolver encapsulada la respuesta producida en la isla remota.	Nada

8.3. Relacion con los mensajes basicos

Mensaje	Quien lo usa	Funcion
REGISTER	Servidor de inventario — Intermediario local	Registrar formalmente un servidor en su isla
UNREGISTER	Servidor de inventario — Intermediario local	Retirar un servidor de su isla
HEARTBEAT	Servidor de inventario — Intermediario local, e Intermediario — Intermediario	Verificar actividad
ROUTE_UPDATE	Intermediario — Intermediario	Propagar cambios del sistema
ROUTE_REQUEST / ROUTE_SNAPSHOT	Intermediario — Intermediario	Sincronizar tabla de rutas
REMOTE_QUERY / REMOTE_RESULT	Intermediario — Intermediario	Resolver en otra isla cualquiera de las cinco consultas

8.4. Eventos que deben propagarse

Evento	Cuando pasa	Mensaje utilizado	Efecto
Alta de servidor de inventario	Un servidor completa REGISTER exitosamente	ROUTE_UPDATE	Los demas intermediarios agregan rutas remotas hacia ese nodo
Retiro voluntario	Un servidor envia UNREGISTER	ROUTE_UPDATE	Los demas invalidan o eliminan esas rutas
Caida detectada	Un servidor deja de responder heartbeat o vence el timeout	ROUTE_UPDATE	Los demas marcan el recurso como inactivo
Cambio de catalogo	Un servidor agrega o elimina categorias	ROUTE_UPDATE	Los demas actualizan sus entradas

Evento	Cuando pasa	Mensaje utilizado	Efecto
Recuperacion de nodo	Un servidor vuelve a estar disponible	ROUTE_UPDATE	Los demas lo vuelven a marcar como activo
Inicio de intermediario nuevo	Un intermediario entra al sistema	ROUTE_REQUEST / ROUTE_SNAPSHOT	El nuevo intermediario sincroniza su estado inicial

8.5. Reglas de funcionamiento

Solo el intermediario que detecta el evento original debe propagarlo.

La informacion propagada representa conocimiento indirecto. Si un intermediario recibe un `ROUTE_UPDATE` indicando que existe un servidor de inventario en otra isla, no debe tratarlo como servidor propio ni iniciar heartbeats directos contra el. Debe registrarlo como recurso remoto alcanzable a traves del intermediario que lo anuncio.

Toda solicitud entre intermediarios debe generar una unica respuesta asociada, manteniendo el mismo `ID_MENSAJE`, exactamente igual que en el resto del protocolo. El mensaje encapsulado dentro de un `REMOTE_QUERY` conserva su propio identificador interno, de modo que la correlacion se mantiene en ambos niveles.

Las consultas remotas de negocio deben resolverse por el intermediario responsable de la isla destino. Si `INT-01` necesita una categoria que pertenece a la isla de `INT-02`, no consulta directo al servidor de inventario remoto; envia `REMOTE_QUERY` a `INT-02`, y este realiza localmente la consulta al servidor correspondiente.

Si un mensaje llega con version invalida, tipo desconocido, formato incorrecto o parametros incompletos, debe responderse con los codigos de error ya definidos.

9. Manejo de la tabla de rutas

La tabla de rutas es la estructura local que mantiene cada servidor intermedio para conocer que categorias estan disponibles en el sistema, en que servidor de inventario se encuentran y por medio de que intermediario deben solicitarse. Su proposito principal es permitir que el intermediario tome decisiones de enrutamiento sin depender de descubrimiento improvisado en cada consulta.

A diferencia de propuestas anteriores, la unidad de enrutamiento es la categoria y no el producto individual. Esta decision reduce de forma considerable el tamano de la tabla, ya que un catalogo de cientos de productos genera unicamente tantas entradas como categorias existan, y coincide con la estructura del almacenamiento, donde cada categoria corresponde a un directorio completo. La consecuencia es que un intermediario sabe que la categoria `comida` vive en cierto nodo, pero no sabe de antemano si el producto `pan` existe dentro de ella; esa determinacion la hace el servidor de inventario al resolver la consulta, y se comunica mediante el mensaje 31.

Cada intermediario usa su tabla de rutas para tres tareas principales. Primero, saber que recursos estan disponibles en su propia isla a partir de los registros `REGISTER` recibidos de los servidores locales. Segundo, mantener conocimiento indirecto sobre recursos de otras islas a partir de mensajes propagados entre intermediarios. Tercero, decidir el camino de resolucion de una solicitud, es decir

intentar primero con recursos locales y, si no estan disponibles, reenviar la consulta al intermediario responsable de la isla remota.

9.1. Estructura propuesta

Campo	Descripcion
Categoria	Nombre de la categoria administrada
ID ServidorInventario (ID SI)	Identificador del servidor que posee esa categoria
ID IntermediarioResponsable (ID Int Res)	Identificador del intermediario responsable de acceder a ese servidor
Tipo Ruta	Local o remota
Estado	Activo, Inactivo o Desconocido
Ultima Actualizacion (UA)	Marca logica o temporal del ultimo cambio conocido
Informacion Origen (IO)	Como se conocio la ruta: REGISTER, ROUTE_UPDATE o ROUTE_SNAPSHOT

La tabla de rutas de un intermediario se construye y actualiza de tres maneras. La primera es mediante **REGISTER**: cuando un servidor de inventario local se registra, el intermediario agrega una entrada local por cada categoria anunciada. La segunda es mediante propagacion de cambios entre intermediarios: cuando llega un **ROUTE_UPDATE**, el intermediario agrega o modifica entradas remotas segun la informacion recibida. En este caso, la ruta no se marca como local sino como remota, y el campo del intermediario responsable queda con el identificador del nodo que controla esa isla. La tercera es mediante sincronizacion inicial: cuando un intermediario entra en operacion y solicita un **ROUTE_SNAPSHOT**, usa esa respuesta para poblar la tabla con el estado conocido del sistema antes de empezar a resolver consultas remotas.

9.2. Reglas

Una entrada local siempre tiene como intermediario responsable al propio intermediario. Una entrada remota nunca debe convertirse en local solo por haber sido anunciada por otro nodo. Una ruta remota no autoriza heartbeats directos contra el servidor de inventario remoto, ya que el control directo sigue perteneciendo al intermediario de su isla. Toda actualizacion remota debe dejar evidencia del origen de la informacion, ya sea **ROUTE_UPDATE** o **ROUTE_SNAPSHOT**, para la depuracion y la bitacora. Si hay varias rutas para la misma categoria, el intermediario escoge primero la local, y entre varias remotas elige la primera activa conocida o la mas reciente.

Si llega un **UNREGISTER**, las rutas asociadas a ese servidor ya no deben quedar disponibles, por lo que se eliminan o se marcan como inactivas. Si un servidor local deja de responder a **HEARTBEAT**, sus rutas deben pasar a inactivo.

9.3. Ejemplo de tabla de rutas

Categoria	ID SI	ID Int Res	Tipo ruta	Estado	UA	IO
ropa	SI-01	INT-01	local	activo	t=1050	REGISTER
comida	SI-01	INT-01	local	activo	t=1050	REGISTER
electronica	SI-03	INT-02	remota	activo	t=1048	ROUTE_UPDATE
juguetes	SI-05	INT-03	remota	inactivo	t=1056	ROUTE_UPDATE

10. Manejo de la bitacora de eventos

La bitacora es el mecanismo mediante el cual cada nodo registra los eventos importantes que ocurren durante su operacion. Su proposito es dejar evidencia del comportamiento del sistema para monitoreo, depuracion y consulta. En esta propuesta, todos los nodos pueden mantener una bitacora local, pero el registro mas importante corresponde al servidor intermedio, ya que este concentra la mayor parte de las decisiones del sistema. La bitacora no cambia la logica del protocolo ni de la tabla de rutas, solamente registra lo que ocurrio.

Cada servidor intermedio debe registrar, como minimo, los eventos relacionados con el registro y retiro de servidores de inventario locales, la deteccion de heartbeat exitoso o fallido, los cambios de estado de rutas locales o remotas, la recepcion y envio de **ROUTE_UPDATE**, la recepcion y envio de consultas remotas entre intermediarios, las consultas locales a servidores de inventario, los errores de protocolo, timeout y nodo no disponible, y las consultas que no pudieron resolverse porque la categoria o el producto no existen.

10.1. Estructura de cada entrada

Campo	Descripcion
ID Evento	Identificador interno del evento
Fecha Hora	Momento en que ocurrio
Nodo	Nodo que registra el evento
Categoria	Tipo general del evento
Evento	Nombre del evento especifico
ID Mensaje	Identificador del mensaje asociado, si aplica
Origen	Nodo origen relacionado
Destino	Nodo destino relacionado
Detalle	Informacion del hecho ocurrido
Resultado	Exito, fallo, timeout o rechazado

10.2. Mensajes para consulta de bitacora

Como se requiere que la bitacora sea consultable en linea, se emplean los mensajes 07 **EVENT_LOG_QUERY** y 08 **EVENT_LOG_RESULT**, ya incluidos en la tabla de tipos. El primero solicita los eventos registrados en un nodo y admite en su cuerpo filtros opcionales por rango de tiempo o por resultado. El segundo devuelve el conjunto de entradas que cumplen con el filtro.

11. Transporte por tipo de mensaje

Se utilizara TCP para todos los mensajes de registro, retiro, actualizacion de rutas, consulta de comandos, consultas de catalogo, consultas de existencias y de ficha de producto, consultas remotas entre intermediarios, respuestas de datos, confirmaciones y errores. Dichas operaciones modifican el estado del sistema o requieren una respuesta confiable y correlacionada mediante el mismo `ID_MENSAJE`; para estas instrucciones se necesita que llegue toda la informacion, sin perder ningun paquete. Esto resulta especialmente relevante en las respuestas de catalogo, cuyo tamano crece con la cantidad de productos de una categoria, asi como en las respuestas del comando `info`, que transportan texto libre y pueden superar el tamano de un datagrama.

El mensaje `HEARTBEAT` se transportara mediante UDP, porque corresponde a una verificacion periodica, breve y tolerante a perdida ocasional. Para evitar falsos positivos, un nodo no se considerara inactivo por la perdida de un unico heartbeat, sino unicamente cuando se acumulen varios heartbeats fallidos consecutivos o se exceda el timeout definido por la implementacion.

Los mensajes `ACK` y de error deberan utilizar el mismo transporte que la solicitud a la que responden. De esta forma, el protocolo mantiene coherencia operativa y evita introducir mecanismos adicionales de confiabilidad en operaciones donde TCP ya los proporciona.

12. Tabla de instrucciones final

ID	Significado	Naturalidad	Transporte	Proposito	Respuesta esperada
01	REGISTER	Solicitud	TCP	Registrar un servidor de inventario.	90/Error
02	UNREGISTER	Solicitud	TCP	Retirar un servidor de inventario.	90/Error
03	HEARTBEAT	Solicitud	UDP	Verificar que un nodo sigue activo.	90/Error
04	ROUTE_UPDATE	Solicitud	TCP	Notificar cambios en rutas o disponibilidad.	90/Error
05	ROUTE_REQUEST	Solicitud	TCP	Solicitar el estado actual de rutas.	06/Error
06	ROUTE_SNAPSHOT	Respuesta	TCP	Devolver el estado de rutas conocidas.	Nada
07	EVENT_LOG_QUERY	Solicitud	TCP	Consultar la bitacora de un nodo.	08/Error
08	EVENT_LOG_RESULT	Respuesta	TCP	Devolver eventos registrados.	Nada
10	HELP_QUERY	Solicitud	TCP	Consultar los comandos disponibles.	11/Error
11	HELP_RESULT	Respuesta	TCP	Devolver la lista de comandos.	Nada

ID	Significado	Natura- leza	Trans- porte	Proposito	Respuesta esperada
12	CATEGORY_QUERY	Solicitud	TCP	Consultar las categorias disponibles.	13/Error
13	CATEGORY_RESULT	Respuesta	TCP	Devolver la lista de categorias.	Nada
14	PRODUCT_QUERY	Solicitud	TCP	Consultar los productos de una categoria.	15,30/Error
15	PRODUCT_RESULT	Respuesta	TCP	Devolver la lista de productos.	Nada
16	STOCK_QUERY	Solicitud	TCP	Consultar la cantidad de un producto.	17,30,31/Error
17	STOCK_RESULT	Respuesta	TCP	Devolver la cantidad en existencia.	Nada
18	INFO_QUERY	Solicitud	TCP	Consultar la ficha completa de un producto.	19,30,31/Error
19	INFO_RESULT	Respuesta	TCP	Devolver cantidad, precio, ubicacion y descripcion.	Nada
30	CATEGORY_NOT_ FOUND	Respuesta	TCP	Indicar que la categoria no existe.	Nada
31	PRODUCT_NOT_ FOUND	Respuesta	TCP	Indicar que el producto no existe.	Nada
40	REMOTE_QUERY	Solicitud	TCP	Encapsular una consulta hacia otra isla.	41/Error
41	REMOTE_RESULT	Respuesta	TCP	Devolver la respuesta de la isla remota.	Nada
90	ACK	Respuesta	TCP	Confirmacion de exito sin contenido.	Nada

13. Casos de prueba

Cuadro 13: Casos de prueba de simulacion

ID	Caso de prueba	Entrada cliente	Mensaje SIQP	Resultado esperado
1	Consultar la ayuda	<code>help</code>	10 → 11	Cliente recibe los cinco comandos con su descripcion en espanol
2	Consultar categorias	<code>inventory</code>	12 → 13	Cliente recibe ropa, comida, electronica, juguetes
3	Productos de categoria valida	<code>have ropa</code>	14 → 15	Cliente recibe camisa, pantalon, zapatos
4	Productos de categoria inexistente	<code>have libros</code>	14 → 30	Cliente recibe “Categoria no encontrada”
5	Consultar existencias	<code>stock comida arroz</code>	16 → 17	Cliente recibe 100
6	Consultar ficha completa	<code>info comida arroz</code>	18 → 19	Cliente recibe 100;\$25.50;Pasillo 1, Estante C;arroz blanco de grano largo
7	Ficha con coma en la ubicacion	<code>info electronica laptop</code>	18 → 19	El valor llega intacto tras deshacer el escape
8	Producto inexistente	<code>info comida pan</code>	18 → 31	Cliente recibe “Producto no encontrado”
9	Producto sin descripcion registrada	<code>info juguetes pelota</code>	18 → 19	La linea conserva los tres separadores y el ultimo campo llega vacio
10	Parametros incompletos	<code>info ropa</code>	18 → 106	Cliente recibe “Parametros invalidos”
11	Comando retirado del protocolo	<code>precio ropa camisa</code>	—	Cliente responde localmente que el comando no existe y sugiere <code>help</code>
12	Formato incorrecto	Mensaje con cinco campos	101	Cliente recibe “Formato invalido”
13	Version no soportada	SIQP/2.0 12 ...	103	Cliente recibe “Version no soportada”

ID	Caso de prueba	Entrada cliente	Mensaje SIQP	Resultado esperado
14	Tipo desconocido	ID_TIP0 igual a 77	105	Cliente recibe “Tipo no soportado”
15	Consulta sin registro previo	Servidor no registrado responde	107	Se rechaza la operacion
16	Nodo caido	Servidor no responde	104	La operacion falla por timeout
17	Falla de almacenamiento	Cadena de bloques rota	109	Cliente recibe “Error de almacenamiento”
18	Salida del sistema	salir	—	Todos los hilos terminan